

# SPRINT 2 - DATA PREPARATION & SPATIAL MODEL

## 1. Sprint 2 in simple words

Sprint 2 moves the team from “we understand the data” to “we can safely use the data”. It has two goals: solve the main data-quality problems identified in Sprint 1, and build/validate the spatial link between Building, Floor, Zone and Device.

The expected outcome is not a perfect dataset. It is a controlled dataset for which the team can explain what was changed, what was kept, what was excluded, and how spatial mappings were obtained.

## 2. Sprint 2 target

| Area           | Goal                                                                 | Minimum evidence                |
|----------------|----------------------------------------------------------------------|---------------------------------|
| Data quality   | Make sampling, reset and missing/invalid treatment explicit.         | Rules + treatment logs + QA     |
| Spatial model  | Connect Building → Floor → Zone → Device where the source permits.   | Mapping table + exceptions      |
| Final baseline | Create a versioned analytical dataset ready for feature engineering. | Clean dataset + readiness check |

## 3. Golden rules

- Do not silently change data.
- Do not guess the meaning of an unknown variable or device.
- Do not delete an unusual value just because it looks strange.
- Keep raw values recoverable whenever possible.
- Every transformation must have a documented rule.
- Every imputed/interpolated value must be flagged.
- Every ambiguous spatial mapping must be flagged for review.
- Use the central data dictionary as the semantic reference.
- The final dataset must be reproducible from the raw inputs.

## US-013 — Define resampling and aggregation rules

EP-03 | 5 SP | Must | Dependency: US-012

### User story

As an analyst, I want documented resampling rules so that energy and environmental signals remain analytically meaningful.

### Why are we doing this?

Raw extracts have different sampling intervals. A blind averaging rule can create misleading energy or environmental values.

### How to do it — practical student version

- Start from the Sprint 1 sampling profile.
- Define the target modelling granularity.
- Classify variables by meaning/type before choosing an aggregation rule.
- Document the aggregation function for each variable type.
- Define when interpolation is allowed and when it is not.
- Test the rules on representative series and compare before/after results.
- Keep raw data unchanged; resampling creates processed data.

### Minimum deliverable

- Resampling specification.
- Aggregation matrix: variable/type → rule.
- Interpolation rules and limits.
- Reproducible script/notebook.

### What to show at Sprint Review

Show the target time grid, aggregation matrix, interpolation rules and one before/after example.

### Do not make this mistake

Do not apply the same aggregation or interpolation rule to every variable without checking its meaning.

## US-014 — Detect device-reset interference

EP-03 | 8 SP | Must | Dependencies: US-009, US-012

### User story

As a data engineer, I want reset-related interference identified so that hardware artefacts do not distort model training.

### Why are we doing this?

Device resets can create artificial jumps or abnormal sequences. A model may learn hardware behaviour instead of real building behaviour.

### How to do it — practical student version

- Identify variables that can show reset behaviour from documentation and observed series.
- Look for documented/statistical reset signatures and abrupt discontinuities.
- Check suspected resets against the surrounding time sequence.
- Create a rule that marks affected intervals.
- Do not overwrite original values.
- Keep a link to the original source.

### Minimum deliverable

- Written reset-detection rule.
- Affected-interval table with device, time and reason.
- Quality flag such as `is_reset`.
- Original-value traceability.

### What to show at Sprint Review

Show a few reset examples, the rule and the affected intervals.

### Do not make this mistake

Do not overwrite or delete suspected reset values before preserving the original.

## US-015 — Treat missing and invalid observations

EP-03 | 8 SP | Must | Dependencies: US-013, US-014

### User story

As an analyst, I want missing/invalid values treated according to documented rules so that the modelling dataset is coherent.

### Why are we doing this?

Missing is not automatically zero, and abnormal is not automatically removable. Repairs and exclusions must remain explainable.

### How to do it — practical student version

- Separate true missing observations from zeros and invalid/anomalous observations.
- Use the time grid to identify missing periods, not only Excel NaN cells.
- Define rules: keep, flag, interpolate/impute, or exclude.
- Only reconstruct values when evidence makes it reasonable.
- Flag every imputed/interpolated value.
- Record a reason for every exclusion.
- Keep original values in the traceability layer.

### Minimum deliverable

- Treatment decision table.
- Treatment log.
- Imputation/interpolation flags.
- Exclusion reasons.

### What to show at Sprint Review

Show examples of missing, invalid and treated observations and explain each decision.

### Do not make this mistake

Do not replace missing values with zero by default.

## US-016 — Produce a clean analytical dataset

EP-03 | 8 SP | Must | Dependencies: US-015, US-022

### User story

As the modelling team, we want a clean, validated dataset so that feature engineering can start from a controlled baseline.

### Why are we doing this?

Feature engineering should not start on an uncontrolled dataset. This task creates the baseline for later sprints.

### How to do it — practical student version

- Run the agreed preparation pipeline.
- Apply timestamp, resampling, reset and missing/invalid rules.
- Apply validated spatial mappings available at this point.
- Run final schema, temporal, missingness and reset QA checks.
- Record dataset version and processing date.
- Publish a short quality summary.
- List unresolved issues and decide whether they block readiness.

### Minimum deliverable

- Versioned clean analytical dataset.
- QA summary.
- Dataset/schema specification.
- Reproducible processing script or pipeline.
- Explicit list of remaining issues.

### What to show at Sprint Review

Demonstrate that the raw data can be processed again to reproduce the controlled dataset.

### Do not make this mistake

Do not call the dataset clean without a final QA check and version.

## US-018 — Map device IDs to zones

EP-04 | 8 SP | Must | Dependencies: US-007, US-017

### User story

As an analyst, I want devices linked to their zones through unique IDs so that telemetry is spatially attributable.

### Why are we doing this?

A consumption value is more useful when we know where it comes from. A stable mapping also prevents inconsistent manual joins later.

### How to do it — practical student version

- Start from the central data dictionary and established unique device IDs.
- Identify available device-to-zone evidence.
- Build one authoritative mapping table.
- Use stable IDs, not file position or unsupported text joins.
- Flag unmapped devices.
- Flag devices with multiple plausible mappings.
- Record the source/provenance of each mapping.

### Minimum deliverable

- Device-to-zone mapping table.
- Fields such as device\_id, device\_family, building, floor, zone, source, confidence/status.
- Unmapped/ambiguous list.
- Mapping provenance.

### What to show at Sprint Review

Pick representative devices and show exactly why each is assigned to its zone.

### Do not make this mistake

Do not create an authoritative mapping from an unsupported manual assumption.

## US-019 — Validate orientation-based zone naming

EP-04 | 3 SP | Must | Dependency: US-017

### User story

As a data-quality owner, I want zone naming conventions validated so that identifiers such as 1NE/1NO are interpreted consistently.

### Why are we doing this?

Zone names contain spatial information. Inconsistent interpretation makes the spatial model unreliable.

### How to do it — practical student version

- Collect zone labels present in the data.
- Identify the naming pattern.
- Parse floor and orientation where the pattern is valid.
- Compare labels with supplied documentation/plans.
- List invalid, unexpected or ambiguous labels.
- Do not silently rename uncertain labels.

### Minimum deliverable

- Naming-convention rule.
- Validated zone-label list.
- Invalid/unexpected label list.
- Simple parsing/validation rule if applicable.

### What to show at Sprint Review

Show examples of valid and invalid labels and explain the rule.

### Do not make this mistake

Do not silently convert an unknown label into a guessed standard label.

## US-020 — Reconcile floor-plan context with dataset mappings

EP-04 | 5 SP | Should | Dependencies: US-018, US-019

### User story

As a reviewer, I want spatial mappings checked against the supplied floor-plan visuals so that the semantic model reflects the documented building structure.

### Why are we doing this?

A spreadsheet mapping alone is not enough if the physical structure says something different. Plans provide an independent check.

### How to do it — practical student version

- Select representative mappings, especially uncertain/important ones.
- Compare them against supplied floor plans.
- Check floor and orientation consistency.
- Record every exception or disagreement.
- Do not infer a physical location from visual proximity alone.
- Escalate unsupported mappings.

### Minimum deliverable

- Cross-check table.
- Exception list.
- Evidence of plan references used.
- Updated mapping status where the source supports a correction.

### What to show at Sprint Review

Show several mappings checked against plans and at least one exception/unresolved case if present.

### Do not make this mistake

Do not infer a device location only because it appears visually close on a plan.

## US-021 — Handle floor-specific HVAC context

EP-04 | 3 SP | Should | Dependency: US-020

### User story

As an analyst, I want floor-specific HVAC context represented where documented so that model features do not assume identical systems across floors.

### Why are we doing this?

HVAC systems may differ by floor. Treating floors as identical can create incorrect features and comparisons.

### How to do it — practical student version

- Collect HVAC information explicitly documented in the source.
- Represent known floor-level HVAC information in the reference table.
- Preserve the third-floor note indicating four HVAC systems.
- Separate known information from unknown topology.
- Do not invent HVAC connections.

### Minimum deliverable

- Floor/HVAC reference table or documented mapping.
- Preserved third-floor context.
- Unknown HVAC topology explicitly flagged.

### What to show at Sprint Review

Show the floor-specific HVAC information and distinguish facts from unknowns.

### Do not make this mistake

Do not invent HVAC topology that is not documented.

## US-022 — Create a spatial validation report

EP-04 | 5 SP | Must | Dependencies: US-018, US-019

### User story

As the team, we want a spatial QA report so that hierarchy and device-to-zone integrity are demonstrable.

### Why are we doing this?

The spatial model is a dependency for later spatial modelling. We need measurable evidence that coverage is sufficient and gaps are visible.

### How to do it — practical student version

- Measure Building → Floor → Zone coverage.
- Count mapped, unmapped and ambiguous devices/records.
- Check naming consistency.
- Check representative mappings against floor-plan evidence.
- Include HVAC context and exceptions where relevant.
- Publish QA evidence with release documentation.
- Conclude whether remaining spatial issues block feature engineering.

### Minimum deliverable

- Spatial validation report.
- Coverage counts/percentages.
- Unmapped and ambiguous lists.
- Key exceptions and evidence.
- Clear conclusion: ready / ready with actions / not ready.

### What to show at Sprint Review

Use the report as the spatial QA gate and explain remaining issues.

### Do not make this mistake

Do not report only a percentage; keep the actual unmapped/ambiguous cases visible.

## 4. Minimum folder structure

- 01\_raw — original source files, never edited
- 02\_dictionary — data dictionary and reference mappings
- 03\_quality — sampling, reset, missingness and treatment reports
- 04\_spatial — device-to-zone mapping, naming checks and plan checks
- 05\_processed — intermediate datasets
- 06\_clean — versioned analytical dataset
- 07\_scripts — reproducible Python code
- 08\_review — Sprint Review evidence

## 5. Sprint 2 success test

At the end of the sprint, the team should be able to answer these questions clearly:

- What time grid are we using, and why?
- How did we treat resets, missing values and invalid observations?
- Can we reproduce the clean dataset from the raw data?
- Can we trace devices to Building → Floor → Zone where the source permits?
- What problems remain, and do any of them block feature engineering?

The Product Backlog defines Gate G1 as passed when no critical unresolved data-quality or spatial-integrity issue blocks feature engineering. The sprint therefore needs evidence, not simply a declaration that tasks are complete.

## 6. Source basis

This guide follows the supplied CeDInt-SEF / DataPack PM<sup>2</sup> Product Backlog and Sprint 2 execution material. Task IDs, priorities, dependencies, story points, user stories and acceptance criteria are preserved from those sources. The practical “how to” sections operationalise the backlog for a student team without changing its scope.